Passing variables via widgets

Widgets pass variables to notebooks similarly to how command-line arguments pass variables to Python scripts. The code block in *Figure 2.14* creates the widgets needed to pass variables from the `Run` command to the `global-setup.py` file using Databricks Utilities or `dbutils`. You can read more about the `dbutils` capabilities in the Databricks Utilities documentation in *Further reading*. These widgets create, pass, and access parameters. The arguments are the variable name, default value, and verbose name in respective order.

```
# RUN TIME ARGUMENTS
dbutils.widgets.text("env", "dev", "Environment")

#ignored if db is set (we force the databse to the given value in this case)
dbutils.widgets.text("project_name", "", "Project Name")

#Empty value will be set to a database scoped to the current user using project_name
dbutils.widgets.text("db", "", "Database")
```

Figure 2.14 – Creating widgets for accepting notebook-specific variables

You can pass each variable while running the file by adding a single line cell with appropriate parameters at the top of the notebook, as shown in *Figure 2.15*.

```
%run ../../global-setup $project_name=synthetic_transactions
```

Figure 2.15 – Running project-specific variables in our global setup file

Running `global-setup.py` saves all the variables defined in the script in memory for easy reference.

Checking for compatibility

Next, in `global-setup.py`, we run checks for compatibility between the code base and the cluster attached to the notebook.

The compatibility code block checks the following:

- A project name was submitted as a variable.
- The cluster is configured with an ML runtime and meets the minimum version. To be sure all features in the code are available in the runtime used, we set a minimum.

Once all checks pass, we assign a user and paths.